

Server-Side Template Injection (SSTI)

Research Report – Detection, Exploitation Concepts, Impact & Defense

Prepared by **Azariya**

INSA Ethio Cyber Talent Weekend Program – Cybersecurity Track

Introduction

Server-Side Template Injection, commonly called SSTI, is a vulnerability that appears when applications treat user input as part of a server-side template instead of ordinary text. Modern frameworks rely on template engines such as Jinja2, Twig, ERB, and Freemarker to generate dynamic pages. When developers accidentally allow user-controlled content to be interpreted by the template engine, attackers may gain access to internal application objects and potentially execute code on the server.

How SSTI Happens

A secure application keeps templates fixed and inserts user data only as variables. Problems begin when applications dynamically construct templates using user-controlled input. In that situation, the rendering engine interprets attacker expressions as active template syntax rather than harmless text. Even small mistakes in template construction can create severe security risks.

Detection and Fingerprinting

Security researchers usually begin SSTI testing with arithmetic expressions. If a payload is evaluated instead of printed literally, the target may be vulnerable. Different engines respond differently to the same payloads, which helps testers identify the underlying technology stack and choose appropriate exploitation methods.

Exploitation Concepts

Once SSTI is confirmed, attackers attempt to interact with internal objects exposed by the template engine. In Python Flask applications using Jinja2, some globally available objects may expose access to Python internals. This can allow attackers to move from simple expression evaluation into operating-system interaction and remote code execution.

Blind SSTI

Some vulnerable applications process template expressions without displaying the result to the user. In these cases, attackers rely on out-of-band techniques such as DNS or HTTP callbacks. If the server contacts an external monitoring system controlled by the tester, the payload execution is confirmed even without visible output on the webpage.

Impact

A successful SSTI attack may expose sensitive configuration files, credentials, API keys, and internal application data. In severe cases, attackers can execute system commands, establish persistence, or pivot into internal infrastructure. Because SSTI frequently results in Remote Code Execution, it is considered a critical vulnerability class.

Defensive Measures

Organizations can reduce SSTI risks by avoiding dynamic template construction, using sandboxed template environments, restricting dangerous objects from template contexts, validating user input, and applying the principle of least privilege. The most effective defense remains strict separation between template logic and user-controlled data.

Conclusion

Server-Side Template Injection demonstrates how unsafe handling of dynamic content can escalate into complete server compromise. Understanding template engine behavior, detection methods, exploitation concepts, and defensive controls is essential for developers, defenders, and penetration testers working with modern web applications.